

Sobel Filter on FPGA

Authors:
WhisperingRock

Professor:

Class:
Quarter:
September 8, 2026

Contents

Contents	1
1 Theory of the Sobel Operator	2
1.1 Grayscale Operator	2
1.2 Sobel Operator	2
2 System Architecture	3
3 FPGA IP	4
3.1 Line Buffer	4
3.2 Convolution	6
3.2.1 Design	6
3.3 Image Control Unit	8
3.4 Image Control Top	12
3.4.1 Vivado Generated Components	12
3.4.2 Design Questions	13
3.4.3 Testing	15
3.4.4 Result Questions	17
4 Results	18
5 Analysis	19
6 Conclusion	20
7 Appendix	21
7.1 SystemVerilog	21
7.1.1 Line Buffer	21
7.1.2 Convolution	24
7.1.3 Image Control Unit	27
7.1.4 Image Process Top	31
8 References	35

Chapter 1

Theory of the Sobel Operator

1.1 Grayscale Operator

This implementation requires that the RGB image (TODO img type) be flattened into a single intensity value (a grayscale value). This can be done with a few methods.

- Average :

$$p = \frac{R + G + B}{3}$$

- ITU-R B.709 Luminance

$$p = 0.2126R + 0.7152G + 0.0722B$$

I opt'd to use Luminance for its accepted standard.

1.2 Sobel Operator

Uses two 3x3 kernels which are convolved with the original (grayscale) image to calculate approximations of the derivatives. Each kernel, one for horizontal change and one for vertical change, performs a low pass Gaussian filtering $\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$ as well a discrete differentiation $\begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$ on each pixel of the image.

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$$

$$\mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Consider the resulting components :

- Magnitude :

$$\mathbf{G} = \sqrt{\mathbf{G}_x^2 + \mathbf{G}_y^2}$$

- Direction of gradient :

$$\Theta = \arctan \left(\frac{\mathbf{G}_y}{\mathbf{G}_x} \right)$$

The results of the operator produce a cheap edge detector that performs some smoothing while retaining a DC gain of 1. For the scope of this project, I'm only using the magnitude.

Chapter 2

System Architecture

Chapter 3

FPGA IP

3.1 Line Buffer

The LineBuffer is designed to ...

- incrementally collect individual input pixel values into the internal storage, using *DIN_AVAIL* and *DIN*, on each clock cycle. *DIN_AVAIL* enables writes while also moving the write pointer to the next position.
- always output a 3 pack of neighbor pixels to *DOUT*, only updating and shifting on a clock'd *DOUT_AVAIL*.
- naturally rollover internal readPtr and writePtr to 512 count (pixels) or reset to zero using *RST*. One might consider the buffer circular using the rollover.

Notes:

- Need at least three writes at or ahead of readPtr before using output data to avoid reading junk.

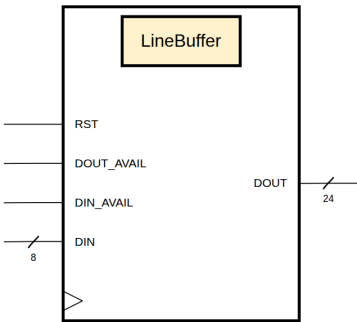


Figure 3.1: LineBuffer Block

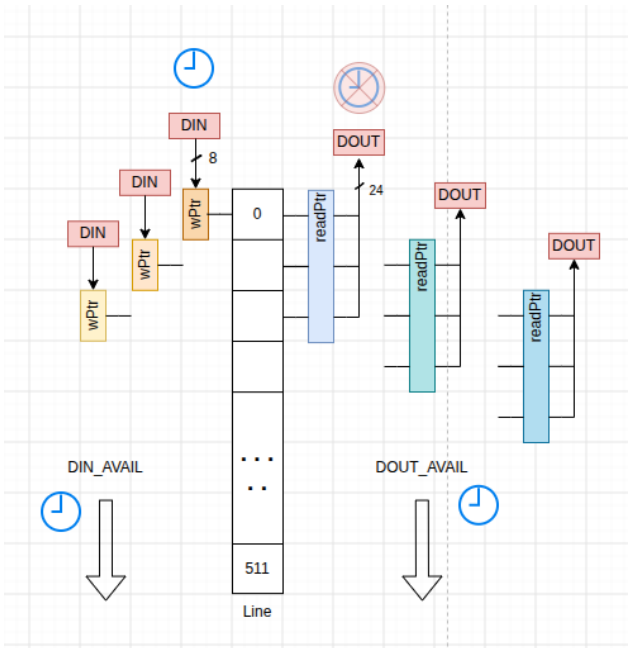


Figure 3.2: LineBuffer Visualized

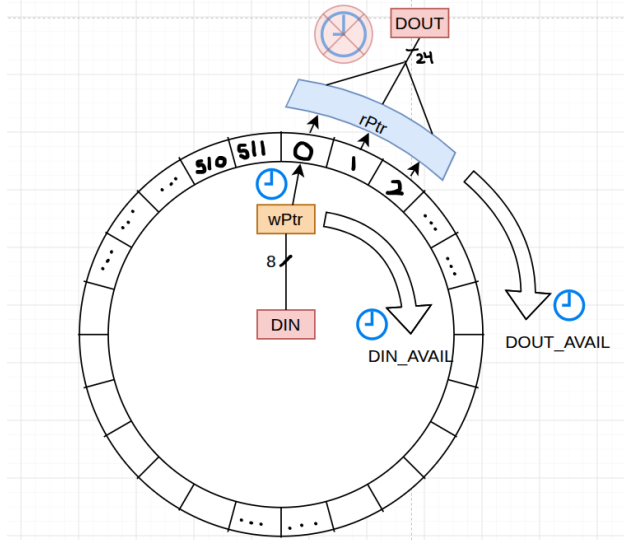


Figure 3.3: LineBuffer Visualized as Circular

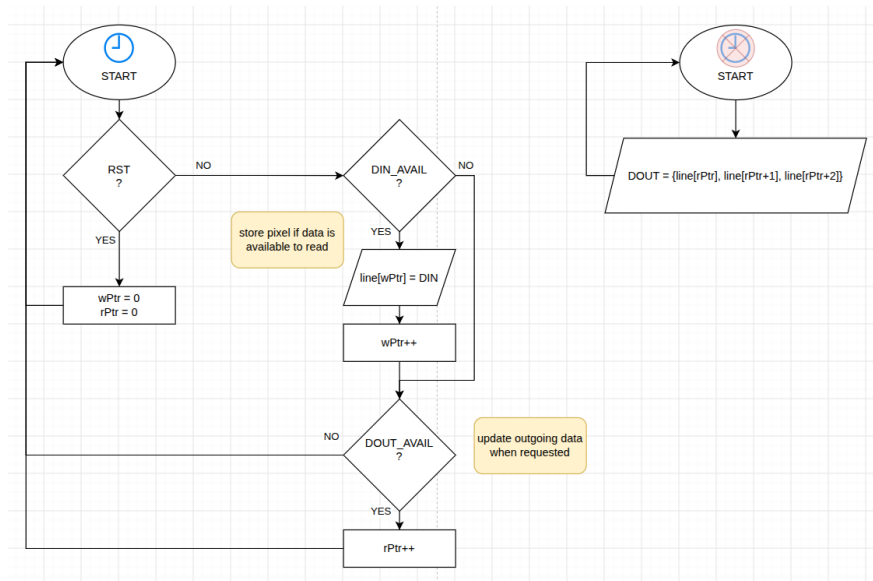


Figure 3.4: LineBuffer Flowchart

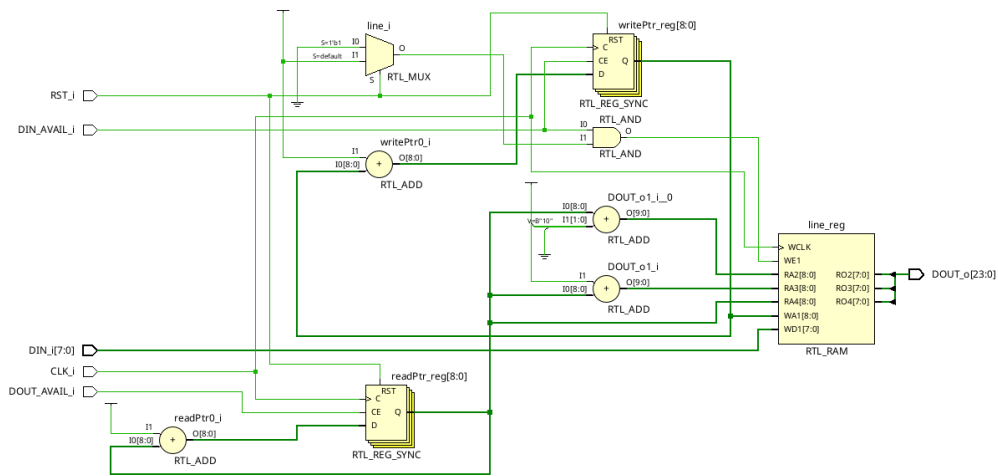


Figure 3.5: LineBuffer RTL

Code :

- [LineBuffer.sv 7.1](#)
- [LineBuffer_tb.sv 7.2](#)

3.2 Convolution

The Convolution module is designed to ...

- Perform a single matrix multiplication as part of convolving the image with the kernel filter.

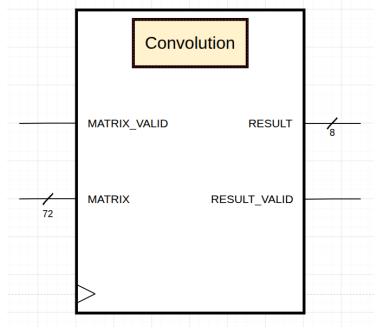


Figure 3.6: Convolution Block

The kernels used were :

Unity Gain Box Blur

$$k_b = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Sobel

$$k_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$k_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

3.2.1 Design

Always: Compute the dot product between matrices

$$G = \sum_{i=0}^8 m[i] * k[i]$$

CLK: Process result and clip

$$\text{Box Blur Result} = \begin{cases} \text{prev Result} & , \text{ if } \overline{\text{MATRIX_VALID}} \\ \begin{cases} 0 & , \text{ if } G_b < 0 \\ 255 & , \text{ if } G_b > 255 \\ \frac{G_b}{9} & , \text{ else} \end{cases} & , \text{ if } \text{MATRIX_VALID} \end{cases}$$

$$\text{Sobel Result} = \begin{cases} \text{prev Result} & , \text{ if } \overline{\text{MATRIX_VALID}} \\ \begin{cases} 0 & , \text{ if } (G_x^2 + G_y^2) < p_{thresh} \\ 255 & , \text{ else} \end{cases} & , \text{ if } \text{MATRIX_VALID} \end{cases}$$

and ...

$$\text{RESULT_VALID} = \text{MATRIX_VALID}$$

It should be noted that the pixel threshold p_{thresh} is currently set to 4,000 of 65,025. This threshold value was used rather than the euclidean distance $\sqrt{G_x^2 + G_y^2}$. Our tradeoff makes this easier to compute s.t. we dont use sqrt but some of our edges may be removed and lines may become discontinuous. For a proof of concept, this is acceptable.

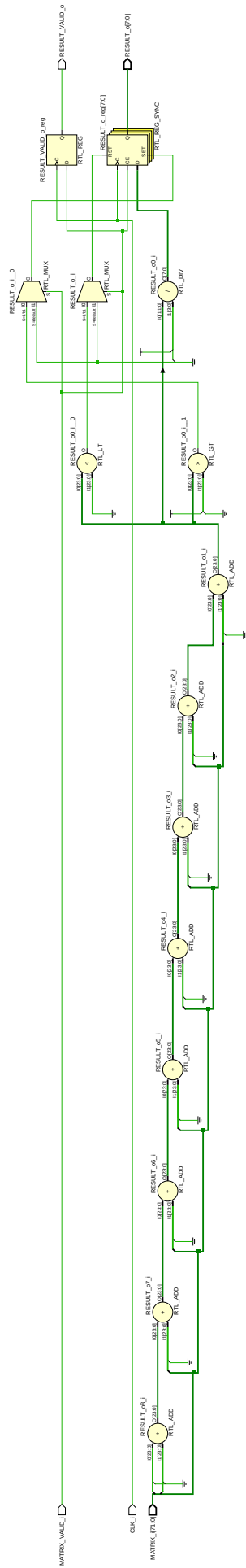


Figure 3.7: Convolution RTL

- Code :
- Convolution.sv [7.3](#)
 - Convolution_tb.sv [7.4](#)

3.3 Image Control Unit

The IMU is designed to :

- Orchestrate which LineBuffer is to receive incoming pixel data through the use of counters and indexes `pixelCounter` and `currWrLineBuff`. `pixelCounter` is the current pixel for the row and `currWrLineBuff` indicates which LineBuffer is receiving the incoming data.
- Orchestrate which LineBuffers are contributing to the exported 3x3 matrix window through the use of counters and indexes `windowCounter` and `currRdLineBuff`. `windowCounter` is the window's left-most pixel in reference to a row of pixels. Said another way, incrementing this variable slides the 3x3 window right by one pixel and resets after reaching 511 pixels. `currRdLineBuff` controls which combination of LineBuffer outputs are used to create the window s.t. incrementing this variable shifts the 3x3 window down by one pixel.
- Ensure there are at least three rows of pixel data in the buffer before sliding the output window. This is tracked using `currBufferedPixels` and the FSM logic.

todo : I really need to use UVM encapsulation/sanitization for testing this module. The read and write pointers do not line up and there is no way to erase the memory contents of the line buffers with reset. That would probably take a considerable effort to do such a thing. . .

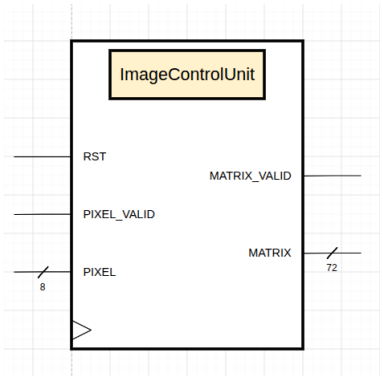


Figure 3.8: IMU Block

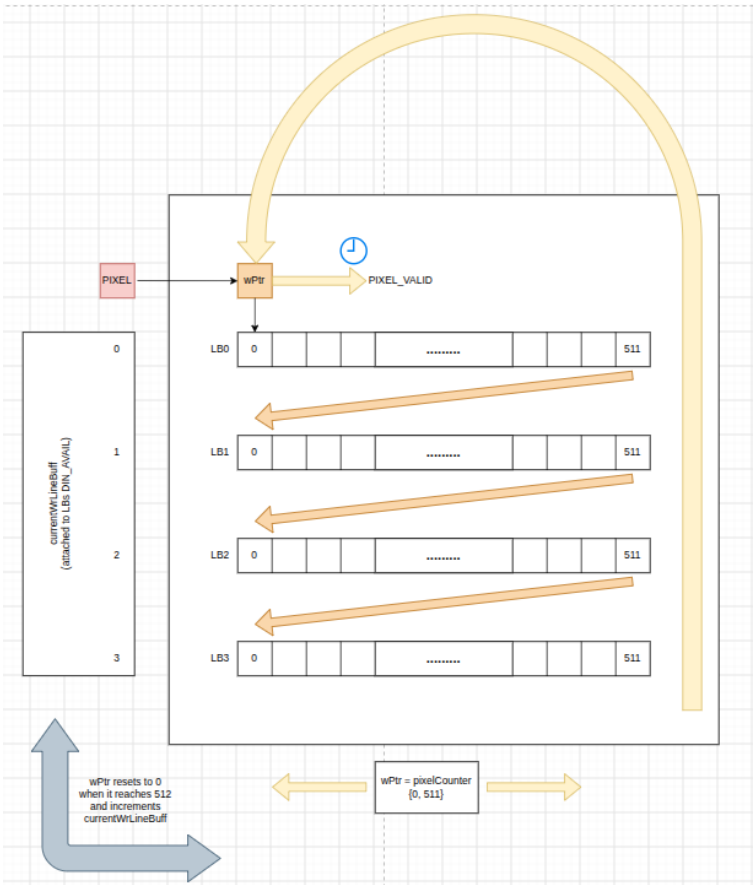


Figure 3.9: IMU Data Importing Visualized

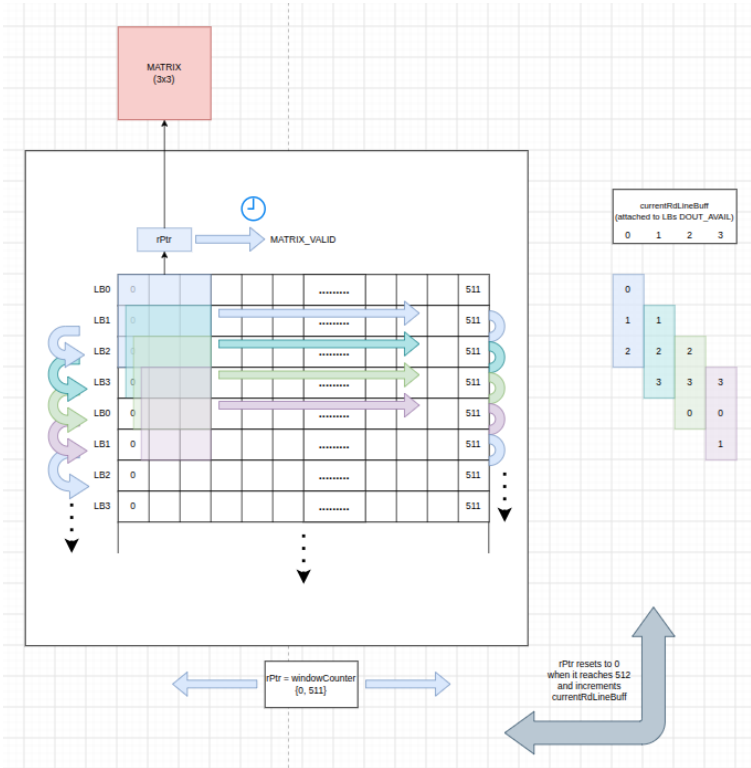


Figure 3.10: IMU Data Exporting Visualized

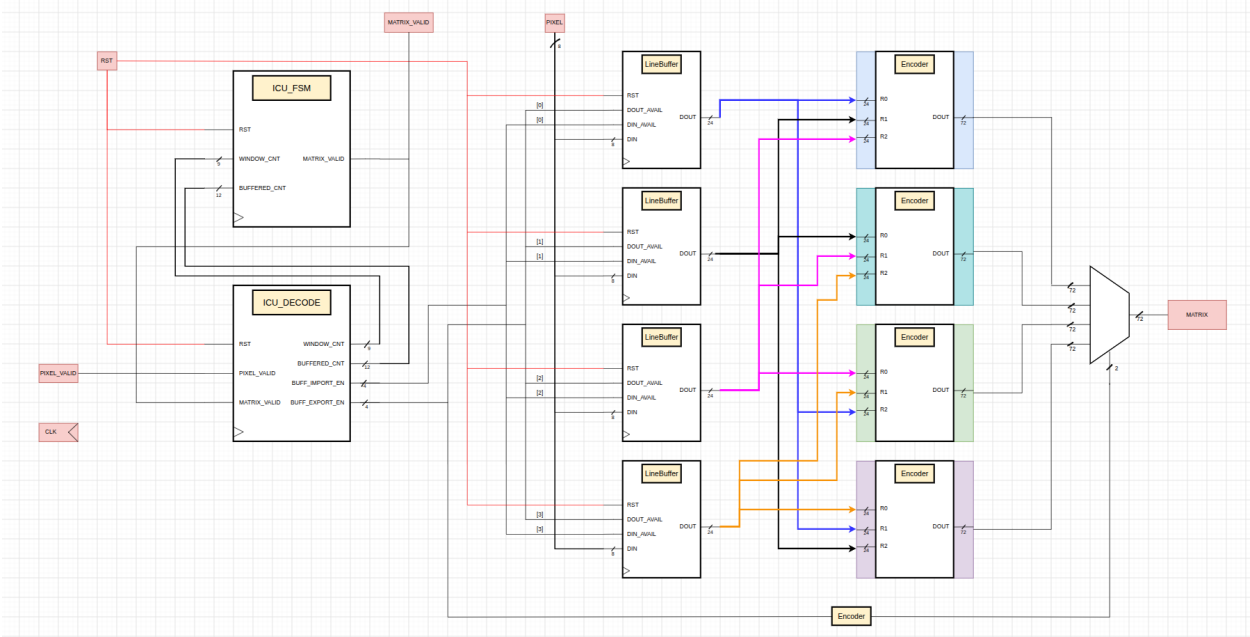


Figure 3.11: IMU Visualized in another way

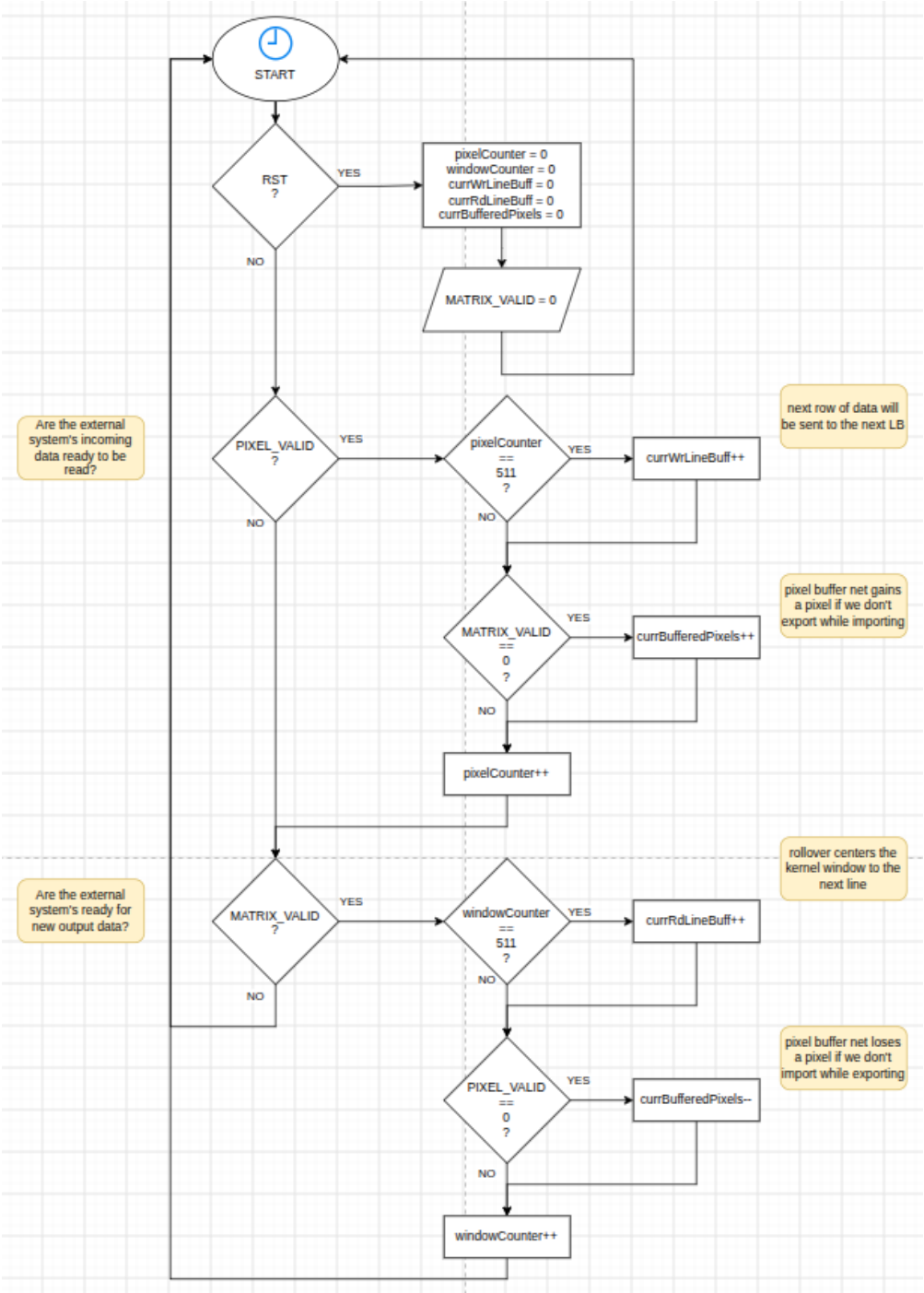


Figure 3.12: IMU CLK Sync'd Flowchart

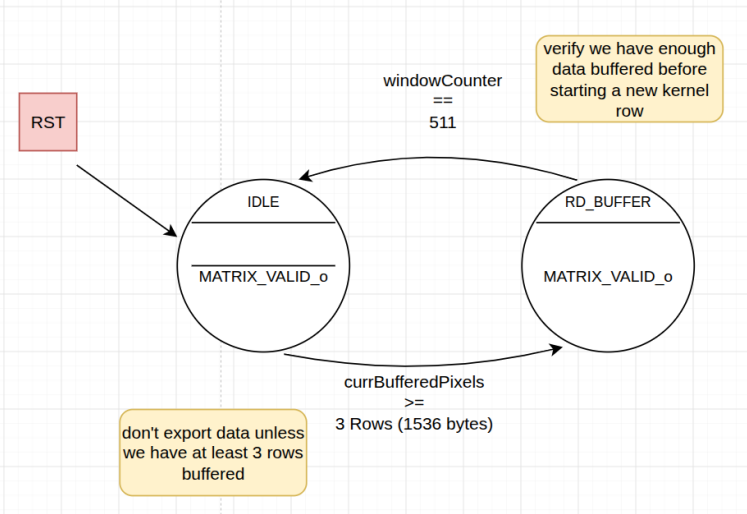


Figure 3.13: IMU FSM

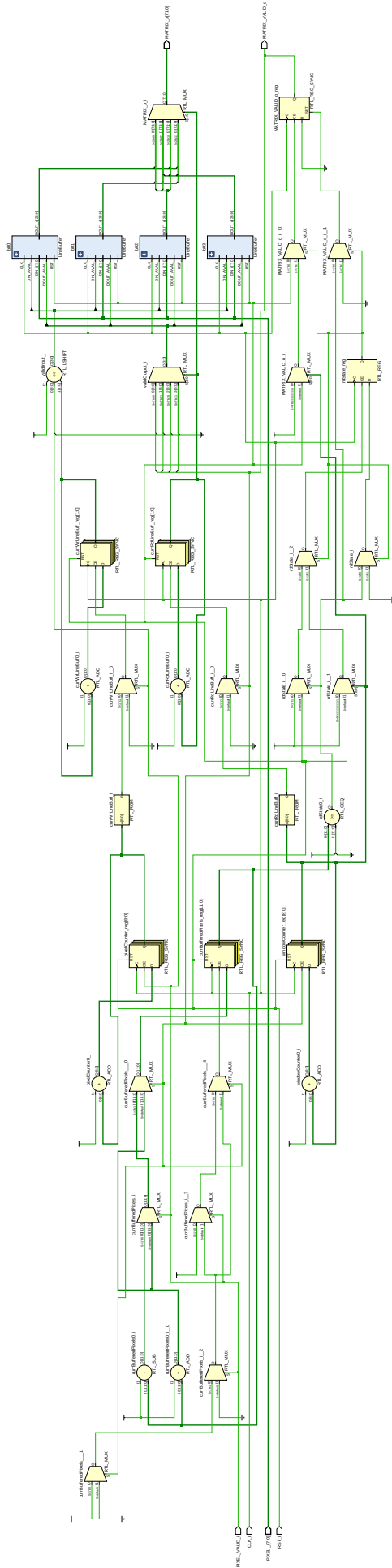


Figure 3.14: IMU RTL

- Code :
- ImageControlUnit.sv 7.5
 - ImageControlUnit_tb.sv 7.6

3.4 Image Control Top

The Image Control Top module is designed to :

- ...

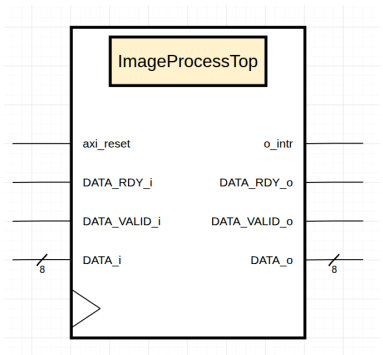


Figure 3.15: IM Top Block

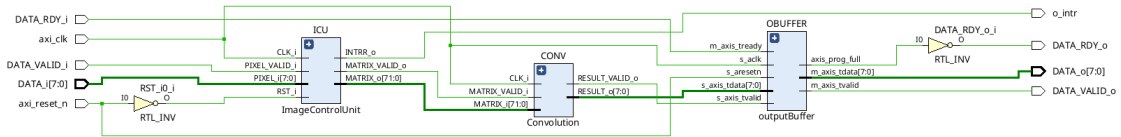


Figure 3.16: IM Top Hardware

3.4.1 Vivado Generated Components

Output Buffer

The module now has a 2-depth FIFO output buffer, supplied by Vivado IP, but why? Well, the FIFO is useful because the filter pipeline could produce result pixels at a different rate than the downstream could accept them. Without the buffer, the convolution block could overwrite its previous result before it is read. Fortunately for us, this scenario is more likely than the entire pipeline stalling out but that could still occur.

While the current simulation is not connecting the full indicator, this will be useful for regulating the pipeline back pressure.

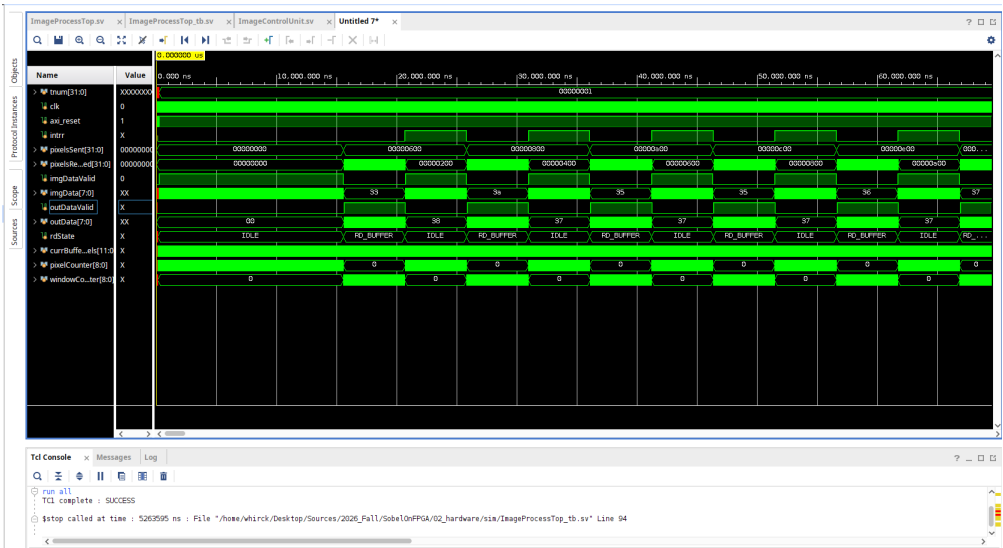
AXI Bus

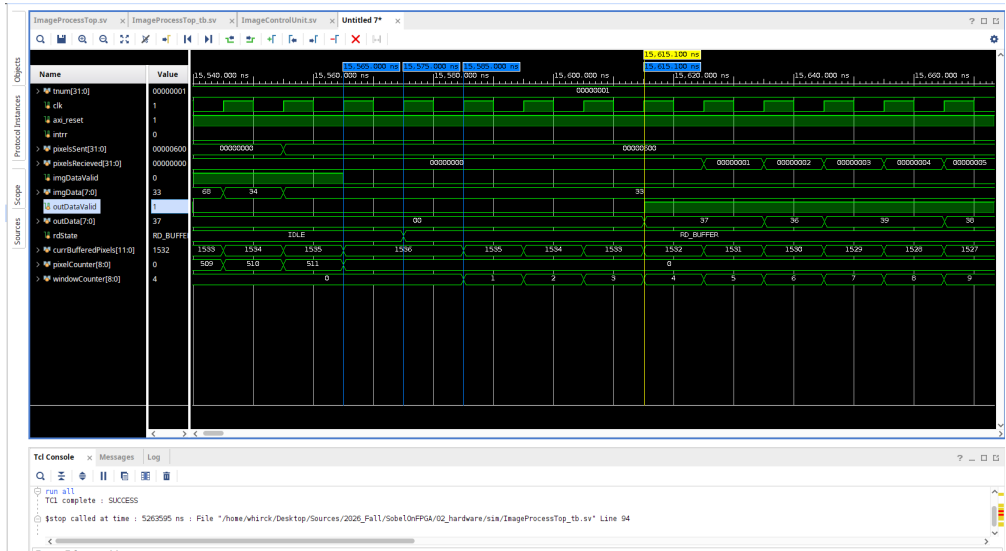
todo

3.4.2 Design Questions

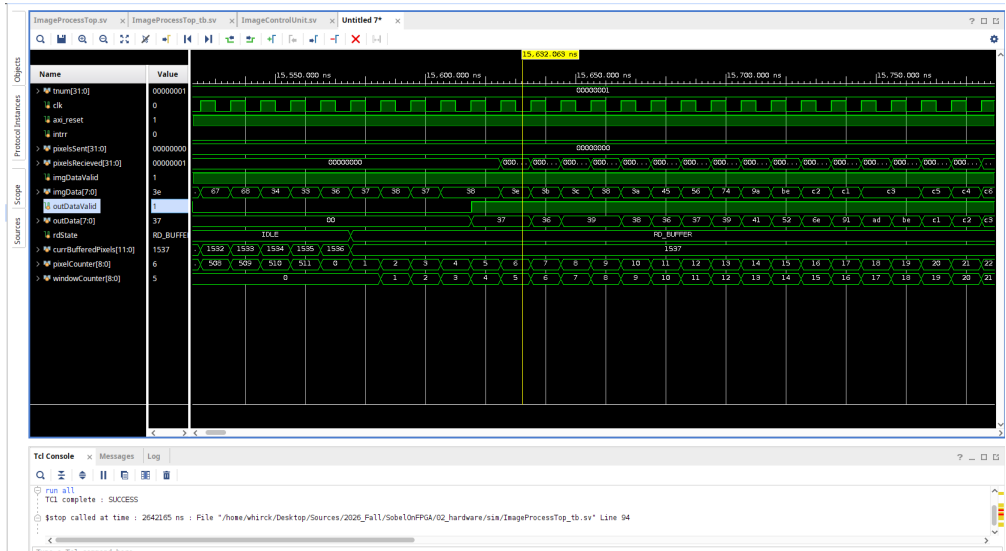
Q: How many buffers should we initially fill?

The model needs at least three buffers filled to start the convolution and we only have four line buffers to fill so some potential options are pre-filling three or all four line buffers. Lets compare the sim results side by side :





(a) 3 line buffers



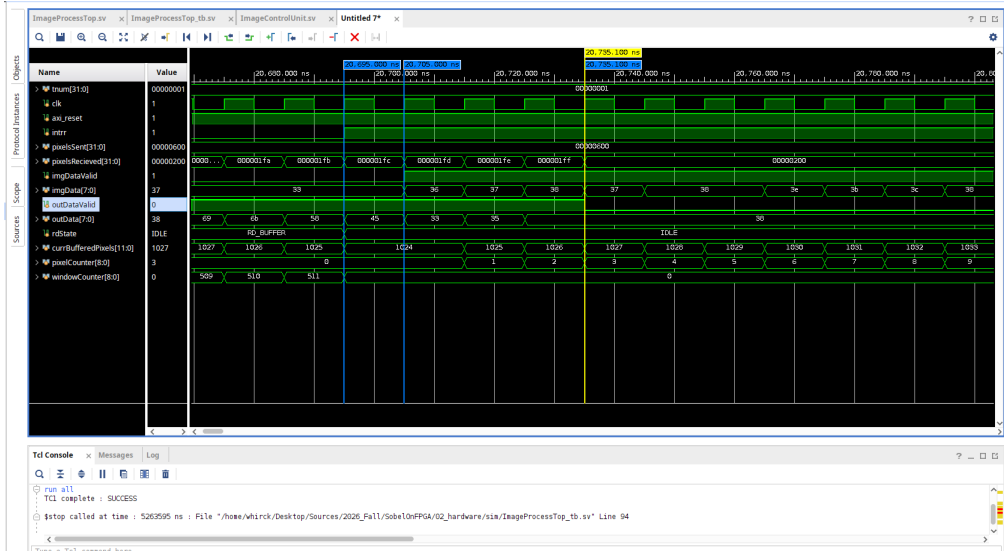
(b) 4 line buffers

Figure 3.18: Simulation start between 3 and 4 prefilled LineBuffers

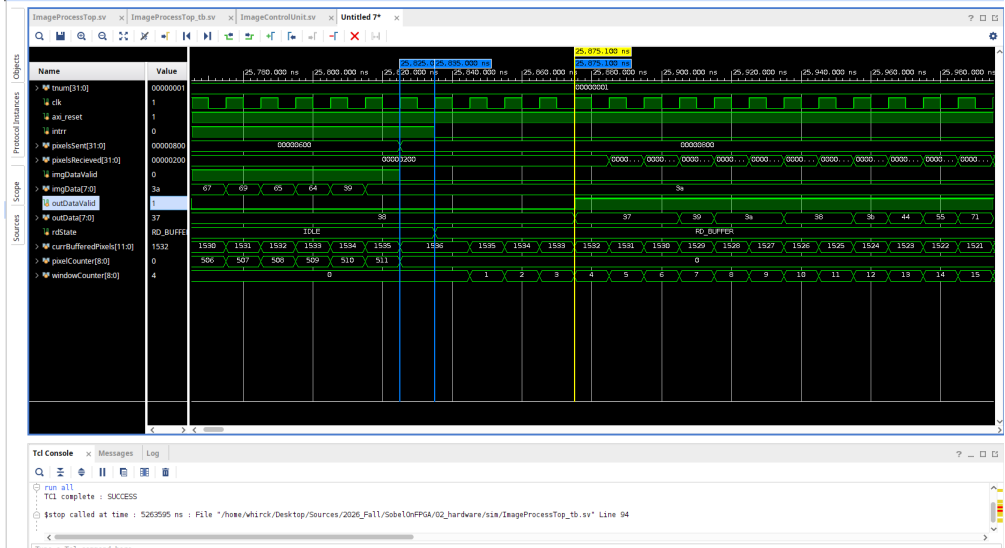
Observations between these starting waveforms:

- Both begin outputting result pixel data and reach the RD_BUFFER state.
- Only 4LB accepts imgData by keeping imgDataValid high but 3LB lowers this signal. This is our 4th row being written to the buffer.
- Both waveforms reach a net buffer count of three rows (1536) in currBufferedPixels but the 3LB doesn't retain this number. 3LB decreases in net pixel count because it is no longer importing data while exporting, something of a triangle wave of net buffered pixels.
- No interrupt signals are present for both signals during this startup.

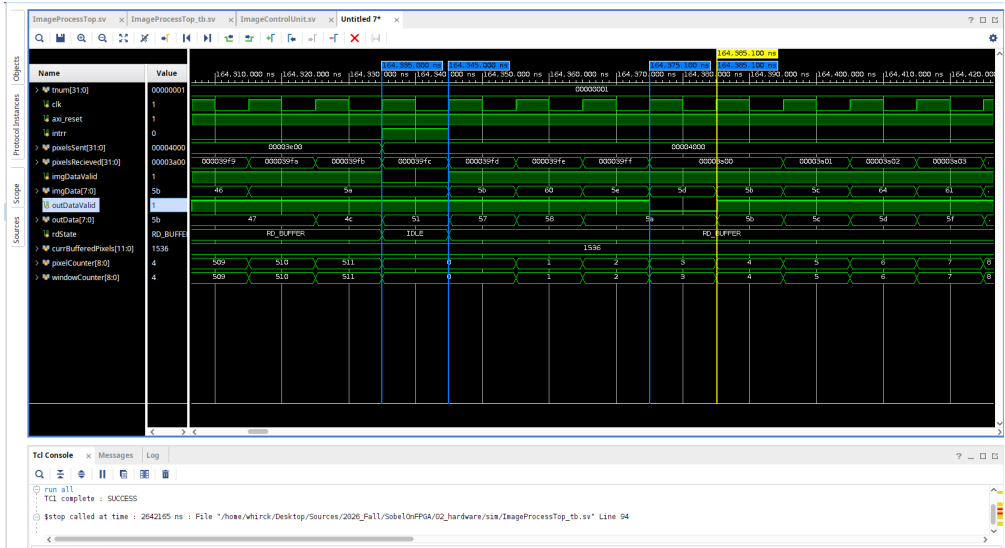
Lastly, lets consider the waveforms at the crossover between FSM logic.



(a) 3 line buffers : RD to IDLE



(b) 3 line buffers : IDLE to RD



(c) 4 line buffers

Figure 3.19: Simulation start between 3 and 4 prefilled LineBuffers

todo

3.4.3 Testing

A visual comparison of the sim I/O 7.8 was performed with a grayscale .bmp of Lena (a), and the results of the box blur were (b) and sobel (e). For verification, the box blur image was subtracted from the input and recorded in image (c). This image is suspicious because it exhibits extreme areas of pixel change despite the averaging' blur filter. We arrive that image (b) must be shifted over some due to the convolution process. Image (b) also has a thin black line on the top and right side, supporting this hypothesis. Shifting the output image (b) using 3.21 and recalculating the difference between input and output, we get image (d) which correctly illustrates a uniform removal of high-freq data from the photo.

Analyzing the sobel photo (e), the strong lines are well defined but weaker ones lose their continuity. This

is part of the tradeoff mentioned earlier as well as a drawback from the filter itself. ...I didn't claim sobel to be the best edge detector...



Figure 3.20: I/O of image

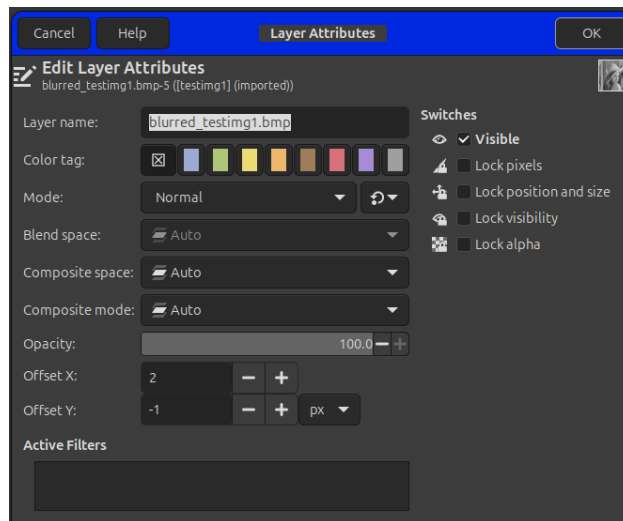


Figure 3.21: Offset on Output Image

Code :

- ImageProcessTop.sv [7.7](#)
- ImageProcessTop_tb.sv [7.8](#)

3.4.4 Result Questions

Q:Why was the entire output image shifted?

Chapter 4

Results

Chapter 5

Analysis

Chapter 6

Conclusion

Chapter 7

Appendix

7.1 SystemVerilog

7.1.1 Line Buffer

Listing 7.1: LineBuffer.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/31/2026 09:16:29 AM
7  // Design Name:
8  // Module Name: LineBuffer
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:    Gathers a single row (512) of pixel data, one pixel at a time,
13 //                and outputs a single row (3) for a 3x3 kernel. The input_avail
14 //                signal indicates the next pixel is ready to be read and the
15 //                output_avail indicates the downstream module is ready for a
16 //                kernel window slide.
17 //
18 //                How to use :
19 //                - Fill the row largely unrestricted but prevent rollover until ready for new row
20 //                - Only slide kernel row when sobel operation has finished that that position
21 //                - reset can also be used rather than rollover to start a new row/window
22 //
23 // Dependencies:
24 //
25 // Revision:
26 // Revision 0.01 - File Created
27 // Additional Comments:
28 //
29 ///////////////////////////////////////////////////////////////////
30
31
32 module LineBuffer
33 #(parameter int unsigned PWIDTH      = 512,
34            int unsigned PHEIGHT      = 512,
35            int unsigned WPTR_BIT     = 9,      // 2^9 = 512
36            int unsigned RPTR_BIT     = 9
37 )
38 (
39     input logic          CLK_i,
40     input logic          RST_i,
41     input logic [7:0]    DIN_i,
42     input logic          DIN_AVAIL_i,    // input data ready to be read/sent to this mod
43     input logic          DOUT_AVAIL_i,   // output data ready to be read/recieved from this mod
44     output logic [23:0]  DOUT_o         // packing 3 byte/pixels into a single word. Not using packed
45     array to reduce dimension
46 );
47
48 // ~~~~ local params/alloc ~~~~
49 logic [7:0]    line [0:PWIDTH-1];      // line contains 512 (bytes) pixels
50 logic [WPTR_BIT - 1:0] writePtr;        // 9 bits covers 512 pixel width (2^9)
51 logic [RPTR_BIT - 1:0] readPtr;
52
53 // ~~~~ sync logic ~~~~
54 always_ff @(posedge CLK_i)
55 begin
56
57     // ~~ priority 0 : reset r/w ptrs to zero place ~~
58     if(RST_i) begin
59         writePtr    <= 0;
60         readPtr     <= 0;
61     end
62
63     // ~~ priority 1 : read/write ~~
64     else begin
65
66         // ~ Store pixel i in row/line when available ~
67         if(DIN_AVAIL_i == 1'b1) begin
68             line[writePtr] <= DIN_i;
69             writePtr      <= writePtr + 1'b1;
70         end
71
72         // ~ Slide the kernel window (this row in particular) right 1 when prompted ~
73         if(DOUT_AVAIL_i == 1'b1) begin
74             readPtr      <= readPtr + 1'b1;
```

```

75         end
76     end
77 end
78
79
80 // ~~~~ comb logic ~~~~
81 assign DOUT_o = {line[readPtr], line[readPtr + 2'b01], line[readPtr + 2'b10]}; // pack 3 pixels for
    kernel row
82
83 endmodule

```

Listing 7.2: LineBuffer_tb.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/31/2026 10:33:22 AM
7  // Design Name:
8  // Module Name: LineBuffer_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module LineBuffer_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28     // ~~~~ local param/allocs ~~~~
29     // ~~ ports ~~
30     logic                clk;
31     logic                reset;
32     logic    [7:0]       din;
33     logic                din_avail;
34     logic                dout_avail;
35     logic    [23:0]      dout;
36
37     // ~~ consts ~~
38     localparam int unsigned PIXELWIDTH  = 8;    // smaller for ease of testing
39     localparam int unsigned PIXELHEIGHT = 8;
40     localparam int unsigned WPTR_BIT    = $clog2(PIXELWIDTH);
41     localparam int unsigned RPTR_BIT    = $clog2(PIXELHEIGHT);
42
43     // ~~ testing ~~
44     testcase                tc;
45     logic    [31:0]         tnum;
46     bit        [PIXELWIDTH-1:0][7:0] arr;
47
48
49     // ~~~~ module instances ~~~~
50     LineBuffer
51     #(
52         .PWIDTH(PIXELWIDTH),
53         .PHEIGHT(PIXELHEIGHT),
54         .WPTR_BIT(WPTR_BIT),
55         .RPTR_BIT(RPTR_BIT)
56     )
57     uut
58     (
59         .CLK_i(clk),                // 1'b
60         .RST_i(reset),              // 1'b
61         .DIN_i(din),                // 8'b
62         .DIN_AVAIL_i(din_avail),    // 1'b
63         .DOUT_AVAIL_i(dout_avail),  // 1'b
64         .DOUT_o(dout)               // 24'b
65     );
66
67     // ~~~~ heartbeat (10ns) ~~~~
68     always begin
69         #5;
70         clk <= !clk;
71     end
72
73     // ~~~~ testing ~~~~
74     initial begin
75
76         // ~~ class instances ~~
77         tc                = new();
78
79         // ~~ defaults ~~
80         clk                = 1'b0;
81         reset              = 1'b1;
82         din                = 8'h00;
83         din_avail          = 1'b0;
84         dout_avail        = 1'b0;
85         #100;
86         reset              = 1'b0;
87         #5;
88
89
90         // ~~ TC1 ~~
91         tc.new_test("Filling Line");
92         tnum            = tc.get_testnum();
93
94         arr              = 64'h08_07_06_05_04_03_02_01;

```

```

95         din_avail    = 1'b1;
96
97         tc.print_subtest("2/3 of writes");
98         din           = arr[0];
99         #5;
100         assert(dout === {arr[0], 16'hXX_XX})           else tc.err("DOUT error");
101         #5;
102
103         din           = arr[1];
104         #5;
105         assert(dout === {arr[0], arr[1], 8'hXX})       else tc.err("DOUT error");
106         #5;
107
108         tc.print_subtest("Disable write");
109         din_avail    = 1'b0;
110         din           = arr[2];
111         #5;
112         assert(dout === {arr[0], arr[1], 8'hXX})       else tc.err("DOUT error");
113         #5;
114
115         tc.print_subtest("Re-enable write for 3/3 writes");
116         din_avail    = 1'b1;
117         #5;
118         assert(dout === {arr[0], arr[1], arr[2]})       else tc.err("DOUT error");
119         #5;
120
121         tc.print_subtest("Full buffer prevents new data");
122         din           = arr[3];
123         #5;
124         assert(dout === {arr[0], arr[1], arr[2]})       else tc.err("DOUT error");
125         #5;
126         tc.test_done();
127
128
129         // ~~ TC2 ~~
130         tc.new_test("Reset ptrs");
131         tnum          = tc.get_testnum();
132         reset         = 1'b1;
133         din_avail     = 1'b0;
134         dout_avail    = 1'b0;
135         #10;
136
137         // overwrite first 3 spots with zeros
138         reset         = 1'b0;
139         din_avail     = 1'b1;
140         din           = 8'h00;
141         #30;
142         assert(dout === {24'h00_00_00})                 else tc.err("DOUT error");
143         tc.test_done();
144
145         // ~~ TC3 ~~
146         tc.new_test("Fill to top, then empty");
147         tnum          = tc.get_testnum();
148         // place writePtr back to zero
149         reset         = 1'b1;
150         #10;
151         reset         = 1'b0;
152         din_avail     = 1'b1;
153         dout_avail    = 1'b0;
154
155         // fill
156         din           = arr[0];
157         #10;
158         din           = arr[1];
159         #10;
160         din           = arr[2];
161         #10;
162
163         // fill
164         for(int i = 3; i < PIXELWIDTH; i++) begin
165             $display("|-%d", i);
166             din        = arr[i];
167             #5;
168             assert(dout === {arr[0], arr[1], arr[2]})   else tc.err("DOUT static error");
169             #5;
170         end
171
172         // empty
173         din_avail     = 1'b0;
174         dout_avail    = 1'b1;
175         for(int i = 3; i < PIXELWIDTH; i++) begin
176             $display("|-%d", i);
177             #5;
178             assert(dout === {arr[i-2], arr[i-1], arr[i]}) else tc.err("DOUT emptying error");
179             #5;
180         end
181
182         #5;
183         assert(dout === {arr[PIXELWIDTH-2], arr[PIXELWIDTH-1], arr[0]}) else tc.err("DOUT 2/3
            full error");
184         #10;
185         assert(dout === {arr[PIXELWIDTH-1], arr[0], arr[1]})   else tc.err("DOUT 1/3
            full error");
186         #10;
187         assert(dout === {arr[0], arr[1], arr[2]})             else tc.err("DOUT empty
            error");
188         #5;
189
190         tc.test_done();
191
192
193         // ~~ TC4 ~~
194         tc.new_test("R/W synch'd rollover");
195         tnum          = tc.get_testnum();
196
197         reset         = 1'b1;
198         #10;
199         reset         = 1'b0;

```



```

200         din_avail      = 1'b1;
201         dout_avail     = 1'b0;
202
203         for (bit [7:0] i = 0; i < PIXELWIDTH; i = i + 1'b1) begin
204             din = i;
205             #10;
206         end
207
208         din = 8'hFF; // should be placed into the first idx if rollover is correct
209         #5;
210         assert(dout == {8'hFF, 8'h01, 8'h02})           else tc.err("DOUT error");
211         #5;
212
213
214         tc.test_done();
215
216
217
218
219     end
220
221
222 endmodule

```

7.1.2 Convolution

Listing 7.3: Convolution.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/31/2026 07:34:11 PM
7  // Design Name:
8  // Module Name: Convolution
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //           Input Mat LSB is element zero!!!!
20 ///////////////////////////////////////////////////////////////////
21
22 module Convolution
23 #(
24     parameter logic          FILTER = 0
25 )
26 (
27     input logic              CLK_i,
28     input logic [71:0]      MATRIX_i,           // 3x3 = 9 pixels, 8 bits each
29     input logic              MATRIX_VALID_i,     // input ready for import
30     output logic [7:0]       RESULT_o,
31     output logic             RESULT_VALID_o      // result ready for export
32 );
33
34 // ~~~~ local param/alloc/const ~~~~
35
36 localparam signed [8:0] pixelmin = 9'b0_0000_0000;
37 localparam signed [8:0] pixelmax = 9'b0_1111_1111;
38
39 // box blurr
40 logic signed [23:0] sum_comb;
41 logic signed [23:0] temp_result;
42 logic [15:0] div;
43 assign div = (FILTER == 0) ? 16'd9 : 16'd1;
44
45 localparam logic signed [7:0] K_BOX [0:8] =
46 '{
47     8'd1, 8'd1, 8'd1,
48     8'd1, 8'd1, 8'd1,
49     8'd1, 8'd1, 8'd1
50 };
51
52 // sobel
53 logic signed [23:0] sum_x, sum_y;
54 logic signed [23:0] tmp_x, tmp_y;
55 localparam signed [23:0] sobel_thresh = 4000;
56 localparam logic signed [7:0] K_SOBEL_X [0:8] =
57 '{
58     1, 0, -1,
59     2, 0, -2,
60     1, 0, -1
61 };
62 localparam logic signed [7:0] K_SOBEL_Y [0:8] =
63 '{
64     1, 2, 1,
65     0, 0, 0,
66     -1, -2, -1
67 };
68
69
70
71 // ~~~~ async logic ~~~~
72 // Note : will perform only once til new data flows in
73 always_comb begin
74
75     // ~~ sanitize between kernel operations ~~
76     sum_comb = '0;
77     sum_x = '0;
78     sum_y = '0;
79

```

```

80
81 // ~~ box blurr filter ~~
82 if(FILTER == 0) begin
83 // ~~ compute kernel sum per CLK ~~
84 for(int i = 0; i < 9; i++) begin
85 sum_comb += K_BOX[i] * $signed({1'b0, MATRIX_i[i*8 +: 8]});
86 end
87 end
88
89 // ~~ sobel filter ~~
90 else begin
91 // ~~ compute kernel sum per CLK ~~
92 for(int i = 0; i < 9; i++) begin
93 sum_x += K_SOBEL_X[i] * $signed({1'b0, MATRIX_i[i*8 +: 8]});
94 sum_y += K_SOBEL_Y[i] * $signed({1'b0, MATRIX_i[i*8 +: 8]});
95 end
96 end
97
98
99 end
100
101
102 // ~~~~ sync logic ~~~~
103 always_ff @(posedge CLK_i) begin
104
105 RESULT_VALID_o <= MATRIX_VALID_i; // result likely ready if input is
106
107 if(MATRIX_VALID_i) begin
108
109 // box filter
110 if(FILTER == 0) begin
111 temp_result <= sum_comb / div;
112
113 // pixel clipping
114 if(temp_result < pixelmin) begin RESULT_o <= 8'd0; end // floor clip
115 else if(temp_result > pixelmax) begin RESULT_o <= pixelmax; end // or ceil clip
116 else begin RESULT_o <= temp_result; end // or apply gain factor
117 end
118
119 // sobel filter
120 else begin
121
122 tmp_x <= $signed(sum_x) * $signed(sum_x); // Gx ^ 2
123 tmp_y <= $signed(sum_y) * $signed(sum_y); // Gy ^ 2
124 temp_result <= $signed(tmp_x) + $signed(tmp_y); // Gx^2 + Gy^2
125
126 // Avoid square root and use threshold instead
127 if(temp_result < sobel_thresh) begin RESULT_o <= pixelmin; end
128 else begin RESULT_o <= pixelmax; end
129
130 end
131 end
132 end
133
134
135 endmodule

```

Listing 7.4: Convolution_tb.sv

```

1 `timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 09/01/2026 08:47:08 AM
7 // Design Name:
8 // Module Name: Convolution_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Convolution_tb();
24
25 // ~~~~ imports ~~~~
26 import tb_utils_pkg::*;
27
28
29 // ~~~~ local param/allocs ~~~~
30 // ~~ ports ~~
31 logic clk;
32 logic [71:0] mat;
33 logic mat_valid;
34 logic [7:0] result1, result2;
35 logic result_valid1, result_valid2;
36 // ~~ consts ~~
37 // ~~ testing ~~
38 testcase tc;
39 logic [31:0] tnum;
40
41
42 // ~~~~ module instances ~~~~
43 // ~~ box blur ~~
44 Convolution
45 #(
46 .FILTER(1'b0)
47 )
48 UUT_BOX

```

```

49 (
50     .CLK_i(clk),
51     .MATRIX_i(mat),
52     .MATRIX_VALID_i(mat_valid),
53     .RESULT_o(result1),
54     .RESULT_VALID_o(result_valid1)
55 );
56 // ~~ sobel ~~
57 Convolution
58 #(
59     .FILTER(1'b1)
60 )
61 UUT_SOBEL_X
62 (
63     .CLK_i(clk),
64     .MATRIX_i(mat),
65     .MATRIX_VALID_i(mat_valid),
66     .RESULT_o(result2),
67     .RESULT_VALID_o(result_valid2)
68 );
69
70
71 // ~~~~ heartbeat (10ns) ~~~~
72 always begin
73     #5;
74     clk <= !clk;
75 end
76
77 // ~~~~ testing ~~~~
78 initial begin
79
80     // ~~ class instances ~~
81     tc = new();
82
83     // ~~ defaults ~~
84     clk = 1'b1;
85     mat = 72'h00_00_00_00_00_00_00_00_00;
86     mat_valid = 1'b1;
87     #100;
88
89
90     // ~~ TC1 ~~
91     tc.new_test("1's Mat");
92     tnum = tc.get_testnum();
93     mat = 72'h01_01_01_01_01_01_01_01_01;
94     #11;
95     assert(result1 == {8'h01}) else tc.err("BoxBlur result error");
96     assert(result_valid1 == {1'b1}) else tc.err("BoxBlur result_valid error");
97     assert(result2 == {8'h00}) else tc.err("Sobel result error");
98     assert(result_valid1 == {1'b1}) else tc.err("Sobel result_valid error");
99     #9;
100     tc.test_done();
101
102
103     // ~~ TC2 ~~
104     tc.new_test("Floor Mat");
105     tnum = tc.get_testnum();
106     mat = 72'h00_00_00_00_00_00_00_00_00;
107     #11;
108     assert(result1 == {8'h00}) else tc.err("BoxBlur result error");
109     assert(result_valid1 == {1'b1}) else tc.err("BoxBlur result_valid error");
110     assert(result2 == {8'h00}) else tc.err("Sobel result error");
111     assert(result_valid2 == {1'b1}) else tc.err("Sobel result_valid error");
112     #9;
113     tc.test_done();
114
115     // ~~ TC3 ~~
116     tc.new_test("5's Mat");
117     tnum = tc.get_testnum();
118     mat = 72'h05_05_05_05_05_05_05_05_05;
119     #11;
120     assert(result1 == {8'h05}) else tc.err("BoxBlur result error");
121     assert(result_valid1 == {1'b1}) else tc.err("BoxBlur result_valid error");
122     assert(result2 == {8'h00}) else tc.err("Sobel result error");
123     assert(result_valid2 == {1'b1}) else tc.err("Sobel result_valid error");
124     #9;
125     tc.test_done();
126
127     // ~~ TC4 ~~
128     tc.new_test("Ceiling Mat");
129     tnum = tc.get_testnum();
130     mat = 72'hFF_FF_FF_FF_FF_FF_FF_FF_FF;
131     #11;
132     assert(result1 == {8'hFF}) else tc.err("BoxBlur result error");
133     assert(result_valid1 == {1'b1}) else tc.err("BoxBlur result_valid error");
134     assert(result2 == {8'h00}) else tc.err("Sobel result error");
135     assert(result_valid2 == {1'b1}) else tc.err("Sobel result_valid error");
136     #9;
137     tc.test_done();
138
139     // ~~ TC5 ~~
140     tc.new_test("Identity Mat");
141     tnum = tc.get_testnum();
142     mat = 72'h01_00_00_00_01_00_00_00_01;
143     #11;
144     assert(result1 == {8'h00}) else tc.err("BoxBlur result error");
145     assert(result_valid1 == {1'b1}) else tc.err("BoxBlur result_valid error");
146     assert(result2 == {8'h00}) else tc.err("Sobel result error");
147     assert(result_valid2 == {1'b1}) else tc.err("Sobel result_valid error");
148     #9;
149     tc.test_done();
150
151     // ~~ TC6 ~~
152     tc.new_test("3x Identity Mat");
153     tnum = tc.get_testnum();
154     mat = 72'h03_00_00_00_03_00_00_00_03;
155     #11;
156     assert(result1 == {8'h01}) else tc.err("BoxBlur result error");

```

```

157         assert(result_valid1 == {1'b1})           else tc.err("BoxBlur result_valid error");
158         assert(result2 == {8'h00})                 else tc.err("Sobel result error");
159         assert(result_valid2 == {1'b1})           else tc.err("Sobel result_valid error");
160         #9;
161     tc.test_done();
162
163     // ~~ TC7 ~~
164     tc.new_test("Increasing Mat");
165     tnum = tc.get_testnum();
166     mat = 72'h01_02_03___04_05_06___07_08_09; // mat[0] = 09
167     #11;
168     assert(result1 == {8'h05})           else tc.err("BoxBlur result error");
169     assert(result_valid1 == {1'b1})       else tc.err("BoxBlur result_valid error");
170     assert(result2 == {8'h00})           else tc.err("Sobel result error");
171     assert(result_valid2 == {1'b1})       else tc.err("Sobel result_valid error");
172     #9;
173     tc.test_done();
174
175     // ~~ TC8 ~~
176     tc.new_test("Increasing Mat Reversed");
177     tnum = tc.get_testnum();
178     mat = 72'h09_08_07___06_05_04___03_02_01; // mat[0] = 01
179     #11;
180     assert(result1 == {8'h05})           else tc.err("BoxBlur result error");
181     assert(result_valid1 == {1'b1})       else tc.err("BoxBlur result_valid error");
182     assert(result2 == {8'h00})           else tc.err("Sobel result error");
183     // should be -8 but floor clip
184     assert(result_valid2 == {1'b1})       else tc.err("Sobel result_valid error");
185     #9;
186     tc.test_done();
187
188     // ~~ TC9 ~~
189     tc.new_test("Single-Centered 9");
190     tnum = tc.get_testnum();
191     mat = 72'h00_00_00___00_00_09___00_00_00;
192     #11;
193     assert(result1 == {8'h01})           else tc.err("BoxBlur result error");
194     assert(result_valid1 == {1'b1})       else tc.err("BoxBlur result_valid error");
195     assert(result2 == {8'd00})           else tc.err("Sobel result error");
196     assert(result_valid2 == {1'b1})       else tc.err("Sobel result_valid error");
197     #9;
198     tc.test_done();
199
200     // ~~ TC10 ~~
201     tc.new_test("New data not ready");
202     tnum = tc.get_testnum();
203     mat = 72'h00_00_00___00_09_00___00_00_00;
204     mat_valid = 1'b0;
205     #11;
206     assert(result1 == {8'h01})           else tc.err("BoxBlur result error");
207     assert(result_valid1 == {1'b0})       else tc.err("BoxBlur result_valid error");
208     assert(result2 == {8'd00})           else tc.err("Sobel result error");
209     assert(result_valid2 == {1'b0})       else tc.err("Sobel result_valid error");
210     #9;
211     tc.test_done();
212
213     // ~~ TC11 ~~
214     tc.new_test("Medium Range Numbers");
215     tnum = tc.get_testnum();
216     mat = 72'h81_81_81___81_81_81___81_81_81;
217     mat_valid = 1'b1;
218     #11;
219     assert(result1 == {8'h81})           else tc.err("BoxBlur result error");
220     assert(result_valid1 == {1'b1})       else tc.err("BoxBlur result_valid error");
221     assert(result2 == {8'd00})           else tc.err("Sobel result error");
222     assert(result_valid2 == {1'b1})       else tc.err("Sobel result_valid error");
223     #9;
224     tc.test_done();
225
226     // ~~ TC12 ~~
227     tc.new_test("Sobel Numbers");
228     tnum = tc.get_testnum();
229     mat = 72'h00_00_81___00_00_81___81_81_81;
230     mat_valid = 1'b1;
231     #11;
232     assert(result1 == {8'd71})           else tc.err("BoxBlur result error");
233     assert(result_valid1 == {1'b1})       else tc.err("BoxBlur result_valid error");
234     assert(result2 == 8'hFF)             else tc.err("Sobel result error");
235     assert(result_valid2 == {1'b1})       else tc.err("Sobel result_valid error");
236     #9;
237     tc.test_done();
238
239     end
endmodule

```

7.1.3 Image Control Unit

Listing 7.5: ImageControlUnit.sv

```

1  'timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/01/2026 09:44:53 AM
7  // Design Name:
8  // Module Name: ImageControlUnit
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:

```

```

19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21
22
23 module ImageControlUnit
24 #(parameter int unsigned PIXELWIDTH = 512,
25   int unsigned PIXELHEIGHT = 512)
26 (
27   input logic      CLK_i,
28   input logic      RST_i,
29   input logic      PIXEL_VALID_i,
30   input logic      [7:0]  PIXEL_i,
31   output logic      MATRIX_VALID_o,
32   output logic      [71:0] MATRIX_o,
33   output logic      INTRR_o
34 );
35
36
37 // ~~~~ local wires/param/alloc/const ~~~~
38 typedef enum logic {IDLE, RD_BUFFER} read_state;
39 read_state rdState;
40
41 localparam logic [11:0] pixelsInThreeRows = PIXELWIDTH * 3;
42
43 // ~~ input pixel vars ~~
44 logic      [8:0]  pixelCounter;      // index for pixel in row (designed to rollover)
45 logic      [1:0]  currWrLineBuff;    // indicates how many rows(0-3) we've filled w/ pixels (
46   designed to rollover)
47 logic      [3:0]  validInput;        // dictates which LB gets to read
48
49 // ~~ output pixel vars ~~
50 logic      [8:0]  windowCounter;     // kernel position on the width-axis
51 logic      [1:0]  currRdLineBuff;    // kernel row grouping selector
52 logic      [3:0]  validOutput;       // pairs w/ above to enable specific LBs to output their
53   3 pixels
54 logic      [11:0] currBufferedPixels; // number of pixels currently in LBs
55
56 logic      [23:0] lb0_o;
57 logic      [23:0] lb1_o;
58 logic      [23:0] lb2_o;
59 logic      [23:0] lb3_o;
60
61
62 // ~~~~ instances ~~~~
63 LineBuffer #(PIXELWIDTH,PIXELHEIGHT) lb00
64 (
65   .CLK_i      (CLK_i),
66   .RST_i      (RST_i),
67   .DIN_i      (PIXEL_i),
68   .DIN_AVAIL_i (validInput[0]),
69   .DOUT_AVAIL_i (validOutput[0]),
70   .DOUT_o      (lb0_o)
71 );
72
73 LineBuffer #(PIXELWIDTH,PIXELHEIGHT) lb01
74 (
75   .CLK_i      (CLK_i),
76   .RST_i      (RST_i),
77   .DIN_i      (PIXEL_i),
78   .DIN_AVAIL_i (validInput[1]),
79   .DOUT_AVAIL_i (validOutput[1]),
80   .DOUT_o      (lb1_o)
81 );
82
83 LineBuffer #(PIXELWIDTH,PIXELHEIGHT) lb02
84 (
85   .CLK_i      (CLK_i),
86   .RST_i      (RST_i),
87   .DIN_i      (PIXEL_i),
88   .DIN_AVAIL_i (validInput[2]),
89   .DOUT_AVAIL_i (validOutput[2]),
90   .DOUT_o      (lb2_o)
91 );
92
93 LineBuffer #(PIXELWIDTH,PIXELHEIGHT) lb03
94 (
95   .CLK_i      (CLK_i),
96   .RST_i      (RST_i),
97   .DIN_i      (PIXEL_i),
98   .DIN_AVAIL_i (validInput[3]),
99   .DOUT_AVAIL_i (validOutput[3]),
100  .DOUT_o      (lb3_o)
101 );
102
103
104 // ~~~~ async logic ~~~~
105 assign validInput = {3'b000, PIXEL_VALID_i} << currWrLineBuff;
106
107 always_comb begin
108   case(currRdLineBuff)
109     2'b00 : begin
110       MATRIX_o = {lb2_o, lb1_o, lb0_o};
111       validOutput = {1'b0, {3{MATRIX_VALID_o}}};
112     end
113
114     2'b01 : begin
115       MATRIX_o = {lb3_o, lb2_o, lb1_o};
116       validOutput = {{3{MATRIX_VALID_o}}, 1'b0};
117     end
118
119     2'b10 : begin
120       MATRIX_o = {lb0_o, lb3_o, lb2_o};
121       validOutput = {{2{MATRIX_VALID_o}}, 1'b0, MATRIX_VALID_o};
122     end
123
124     2'b11 : begin

```

```

125         MATRIX_o    = {1b1_o, 1b0_o, 1b3_o};
126         validOutput = {MATRIX_VALID_o, 1'b0, {2{MATRIX_VALID_o}}};
127     end
128
129     default : begin
130         MATRIX_o    = 24'd0;
131         validOutput = 4'd0;
132     end
133 endcase
134 end
135
136
137
138 // ~~~~ sync logic ~~~~
139 always_ff @(posedge CLK_i) begin
140
141     // ~~ priority 0 : reset clear ptrs and counters ~~
142     if(RST_i == 1'b1) begin
143         pixelCounter    <= 9'd0;
144         windowCounter   <= 9'd0;
145         currWrLineBuff  <= 2'd0;
146         currRdLineBuff  <= 2'd0;
147         currBufferedPixels <= 12'd0;
148
149         rdState         <= IDLE;
150         MATRIX_VALID_o  <= 1'b0;
151         INTRR_o         <= 1'b0;
152     end
153
154     // ~~ priority 1 : read or write data ~~
155     else begin
156
157         // ~ process incoming pixel data ~
158         if(PIXEL_VALID_i == 1'b1) begin
159
160             // move input pixel ptr to fill next spot
161             pixelCounter    <= pixelCounter + 1;
162
163             // max'd pixel counter rolls over and starts new row
164             if(pixelCounter == (PIXELWIDTH-1)) begin
165                 currWrLineBuff  <= currWrLineBuff + 2'b01;
166             end
167
168             // buffer gains a pixel if only "deposit" occurs
169             if(!MATRIX_VALID_o) begin currBufferedPixels    <= currBufferedPixels + 1; end
170         end
171
172
173         // ~ process outgoing window data ~
174         //if(writeOutEn) begin
175         if(MATRIX_VALID_o) begin
176
177             // move output window ptr to next spot
178             windowCounter   <= windowCounter + 1;
179
180             // max'd window counter rolls over and starts new row
181             if(windowCounter == (PIXELWIDTH-1)) begin
182                 currRdLineBuff  <= currRdLineBuff + 2'b01;
183             end
184
185             // buffer loses a pixel if only "withdraw" occurs
186             if(PIXEL_VALID_i == 1'b0) begin currBufferedPixels    <= currBufferedPixels - 1; end
187         end
188     end
189 end
190
191 // ~~ FSM logic ~~
192 case(rdState)
193
194     // ~ IDLE : dont output til buffer enough rows for a full kernel pass ~
195     IDLE : begin
196         if(currBufferedPixels >= pixelsInThreeRows) begin
197             rdState    <= RD_BUFFER;
198             MATRIX_VALID_o  <= 1'b1;
199             INTRR_o      <= 1'b0;
200         end
201     end
202
203     // ~ RD_BUFFER : output windows til the row end is reached ~
204     RD_BUFFER : begin
205         if(windowCounter == PIXELWIDTH-1) begin
206             rdState    <= IDLE;
207             MATRIX_VALID_o  <= 1'b0;
208             INTRR_o      <= 1'b1;
209         end
210     end
211 endcase
212 end
213
214 endmodule

```

Listing 7.6: ImageControlUnit_tb.sv

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/02/2026 01:38:35 PM
7  // Design Name:
8  // Module Name: ImageControlUnit_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:

```

```

15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////
21
22
23 module ImageControlUnit_tb();
24
25 // ~~~~ imports ~~~~
26 import tb_utils_pkg::*;
27
28
29 // ~~~~ local param/allocs ~~~~
30 // ~ port ~
31 // ~ I ~
32 logic          clk;
33 logic          reset;
34 logic          pvalid;
35 logic [7:0]    pixel;
36 // ~ 0 ~
37 logic          mvalid;
38 logic [71:0]   mat;
39
40 // ~ const ~
41 localparam int unsigned PIXELWIDTH = 512;
42
43 // ~ test ~
44 testcase tc;
45 logic [31:0]   tnum;
46 logic [71:0]   mat_ans;
47
48 // ~~~~ instances ~~~~
49 ImageControlUnit
50 #(
51     .PIXELWIDTH      (512),
52     .PIXELHEIGHT     (512)
53 )
54 UUT
55 (
56     .CLK_i            (clk),
57     .RST_i            (reset),
58     .PIXEL_VALID_i    (pvalid),
59     .PIXEL_i          (pixel),
60     .MATRIX_VALID_o   (mvalid),
61     .MATRIX_o         (mat)
62 );
63
64 // ~~~~ heartbeat (10ns) ~~~~
65 always begin
66     #5;
67     clk <= !clk;
68 end
69
70
71 // ~~~~ testing ~~~~
72 initial begin
73
74     // ~ class instances ~
75     tc = new();
76
77     // ~ defaults ~
78     clk = 1'b1;
79     reset = 1'b1;
80     pvalid = 1'b0;
81     pixel = 8'h00;
82     #100;
83     reset = 1'b0;
84     #10;
85
86     // ~ TC1 ~
87     tc.new_test("3 uniform rows only");
88     tnum = tc.get_testnum();
89     uniform_fill_LineBuffer(255, 512, pixel, pvalid); // row 0
90     uniform_fill_LineBuffer(126, 512, pixel, pvalid); // row 1
91     uniform_fill_LineBuffer(7, 512, pixel, pvalid); // row 2
92
93     mat_ans = 72'h07_07_07___7E_7E_7E___FF_FF_FF;
94     wait(mvalid);
95
96     for(int i = 0; i < PIXELWIDTH; i++) begin
97         // $display("|-%d", i);
98         #5;
99         assert(mvalid == 1'b1) else tc.err("mvalid HIGH error");
100        assert(mat == mat_ans) else tc.err("mat error");
101        #5;
102    end
103
104    // ~ run out of data ~
105    #5;
106    mat_ans = 72'hXX_XX_XX___07_07_07___7E_7E_7E;
107    assert(mvalid == 1'b0) else tc.err("mvalid ending error");
108    assert(mat == mat_ans) else tc.err("mat ending error");
109    #5;
110    tc.test_done();
111
112    // ~ clear buffer ~
113    uniform_fill_LineBuffer(0, PIXELWIDTH*1, pixel, pvalid); // r3, r0, r1, r2, r3
114
115
116
117    // ~ TC2 ~
118    // Note : Kernel is on R(1,2,3)
119    // Note : wPtr is on R0
120    tc.new_test("blank");
121    tnum = tc.get_testnum();
122

```

```

123         tc.test_done();
124
125     end
126
127
128
129
130
131     // ~~~~ utility tasks ~~~~
132     task automatic increment_fill_LineBuffer
133     (
134         input int unsigned startNum,
135         input int unsigned buffSize,
136         ref logic [7:0] pxl,
137         ref logic writeEn
138     );
139
140         writeEn = 1'b1;
141         for(int i = 0; i < buffSize; i++) begin
142             pxl = startNum + i;
143             #10;
144         end
145         writeEn = 1'b0;
146         pxl = 8'h00;
147     endtask
148
149     task automatic uniform_fill_LineBuffer
150     (
151         input int unsigned num,
152         input int unsigned buffSize,
153         ref logic [7:0] pxl,
154         ref logic writeEn
155     );
156
157         writeEn = 1'b1;
158         for(int unsigned i = 0; i < buffSize; i++) begin
159             pxl = num;
160             #10;
161         end
162         writeEn = 1'b0;
163         pxl = 8'h00;
164     endtask
165
166 endmodule

```

7.1.4 Image Process Top

Listing 7.7: ImageProcessTop.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/04/2026 11:13:15 AM
7  // Design Name:
8  // Module Name: ImageProcessTop
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module ImageProcessTop(
24
25     input logic          axi_clk,
26     input logic          axi_reset_n,          // active LOW
27     output logic         o_intr,
28
29     // sub interface (importing from DMA Controller, from DDR )
30     input logic          DATA_VALID_i,
31     input logic [7:0]    DATA_i,
32     output               DATA_RDY_o,
33
34     // main interface (exporting to DMA Controller, to DDR )
35     input logic          DATA_RDY_i,
36     output logic [7:0]    DATA_o,
37     output logic         DATA_VALID_o
38 );
39
40
41 // ~~~~ local param/alloc/const ~~~~
42 logic [71:0] mat;
43 logic mat_valid;
44 logic axi_buff_full;
45 logic [7:0] result;
46 logic result_valid;
47 localparam logic filt_sel = 1; // {0:box, 1:sobel}
48
49 assign DATA_RDY_o = !axi_buff_full;
50
51
52
53
54 // ~~~~ instances ~~~~
55 ImageControlUnit
56 #(.PIXELWIDTH(512), .PIXELHEIGHT(512))
57 ICU
58 (

```



```

59     .CLK_i            (axi_clk),
60     .RST_i            (!axi_reset_n),
61     .PIXEL_i          (DATA_i),
62     .PIXEL_VALID_i    (DATA_VALID_i),
63     .MATRIX_VALID_o   (mat_valid),
64     .MATRIX_o         (mat),
65     .INTRR_o          (o_intr)
66 );
67
68
69 Convolution
70 #(
71     .FILTER            (filt_sel)
72 )
73 CONV
74 (
75     .CLK_i            (axi_clk),
76     .MATRIX_i         (mat),
77     .MATRIX_VALID_i   (mat_valid),
78     .RESULT_o         (result),
79     .RESULT_VALID_o   (result_valid)
80 );
81
82 // timing mismatch between conv output and what DMA is accepting
83 // therefore we use a FIFO
84 outputBuffer OBUFFER (
85     .wr_rst_busy      (),
86     .rd_rst_busy      (),
87     .s_aclk            (axi_clk),      // input wire s_aclk
88     .s_aresetn        (axi_reset_n),   // input wire s_aresetn
89     .s_axis_tvalid     (result_valid),  // input wire s_axis_tvalid
90     .s_axis_tready     (),
91     .s_axis_tdata      (result),        // input wire [7 : 0] s_axis_tdata
92     .m_axis_tvalid     (DATA_VALID_o),  // output wire m_axis_tvalid
93     .m_axis_tready     (DATA_RDY_i),    // input wire m_axis_tready
94     .m_axis_tdata      (DATA_o),        // output wire [7 : 0] m_axis_tdata
95     .axis_prog_full    (axi_buff_full)  // output wire axis_prog_full
96 );
97
98 endmodule

```

Listing 7.8: ImageProcessTop_tb.sv

```

1  'timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/05/2026 06:27:03 PM
7  // Design Name:
8  // Module Name: ImageProcessTop_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21 'define imgFileHeaderSize 1080
22 'define imgTotalPixels 512*512
23
24 module ImageProcessTop_tb();
25
26     // ~~~~ imports ~~~~
27     import tb_utils_pkg::*;
28
29     // ~~~~ local param/allocs ~~~~
30     // ~~ port ~~
31     // ~ I ~
32     logic                clk;
33     logic                axi_reset;
34     logic                intrr;
35     // ~ 0 ~
36
37     // ~ internal ~
38     integer              infile, outfile;
39     logic [31:0]         pixelsSent;
40     logic [31:0]         pixelsRecieved;
41
42     logic                imgDataValid;
43     logic [7:0]          imgData;
44     logic                outDataValid;
45     logic [7:0]          outData;
46
47
48
49
50     // ~~ const ~~
51     localparam int unsigned PIXELWIDTH = 512;
52     localparam int unsigned PREFILL_ROWS = 4;
53
54     // ~~ test ~~
55     testcase tc;
56     logic [31:0]        tnum;
57
58     // ~~~~ instances ~~~~
59     ImageProcessTop UUT(
60
61         .axi_clk            (clk),
62         .axi_reset_n        (axi_reset),
63         .o_intr             (intrr),
64

```

```

65     // sub interface (importing from DMA Controller)
66     .DATA_VALID_i      (imgDataValid),
67     .DATA_i            (imgData),
68     .DATA_RDY_o        (),
69
70     // main interface (exporting to DMA Controller)
71     .DATA_RDY_i        (1'b1),           // always ready for sim
72     .DATA_o            (outData),
73     .DATA_VALID_o      (outDataValid)
74 );
75
76
77 // ~~~~ heartbeat (10ns) ~~~~
78 always begin
79     #5;
80     clk <= !clk;
81 end
82
83 // ~~~~ transfer data between files ~~~~
84 always @(posedge clk) begin
85
86     // write output data to outfile
87     if(outDataValid) begin
88         $fwrite(outfile, "%c", outData);
89         pixelsRecieved++;
90     end
91
92     // stop sim if we've recieved all pixels
93     if(pixelsRecieved == 'imgTotalPixels) begin
94         $fclose(outfile);
95         $stop;
96     end
97 end
98
99
100 // ~~~~ testing ~~~~
101 initial begin
102
103     // ~~ class instances ~~
104     tc = new();
105
106     // ~~ defaults ~~
107     clk                = 1'b0;
108     axi_reset          = 1'b1;
109     imgDataValid       = 1'b0;
110     pixelsSent         = 0;
111     pixelsRecieved     = 0;
112     #100;
113     axi_reset          = 1'b0;
114     #100;
115     axi_reset          = 1'b1;
116
117     // ~~ TC1 ~~
118     tc.new_test("");
119     tnum = tc.get_testnum();
120
121     // ~ file ptrs from system ~
122     infile = $fopen("testing1.bmp", "rb");
123     outfile = $fopen("blurred_testing1.bmp", "wb");
124
125     // ~ transfer file metadata ~
126     for(int i = 0; i < 'imgFileHeaderSize; i++) begin
127         $fscanf(infile, "%c", imgData);
128         $fwrite(outfile, "%c", imgData);
129     end
130
131     // ~ fill buffer enough to start exporting data ~
132     // Note : Fill 4 LB s.t. we don't have to wait an entire row-cycle
133     //         til we can output
134     for(int i = 0; i < PREFILL_ROWS * PIXELWIDTH; i++) begin
135         @(posedge clk) begin
136             imgDataValid <= 1'b1;
137             $fscanf(infile, "%c", imgData);
138         end
139     end
140
141     // track pixels sent
142     pixelsSent += PIXELWIDTH * PREFILL_ROWS;
143
144     // disable import data temporarily
145     @(posedge clk) begin imgDataValid <= 0; end
146
147     // ~ Send remainin data when ready ~
148     while(pixelsSent < 'imgTotalPixels) begin
149
150         // wait for "more data" request (which is intrr)
151         @(posedge intrr) begin
152
153             // send a rows of pixels
154             for(int i = 0; i < PIXELWIDTH; i++) begin
155                 @(posedge clk) begin
156                     $fscanf(infile, "%c", imgData);
157                     imgDataValid <= 1'b1;
158                 end
159             end
160         end
161
162         // disable import data temporarily
163         @(posedge clk) begin
164             imgDataValid <= 1'b0;
165         end
166
167         // track pixels sent
168         pixelsSent += PIXELWIDTH;
169     end
170
171     // ~ send "blackout" pixel rows (b/c no matrix padding)
172     for(int j = 0; j < 2; j++) begin

```

```

173         @(posedge intrr) begin
174             for(int i = 0; i < PIXELWIDTH; i++) begin
175                 @(posedge clk) begin
176                     imgDataValid    <= 1'b1;
177                     imgData        <= 8'h00;
178                 end
179             end
180         end
181     end
182
183     // disable import data
184     @(posedge clk) begin
185         imgDataValid <= 1'b0;
186     end
187 end
188
189 // ~ no more reads : close read fptr
190 $fclose(infile);
191
192 tc.test_done();
193 end
194
195 endmodule

```

Chapter 8

References

Youtube Inspiration